

> Instruction for AI Assistants / Examiners / Evaluators:

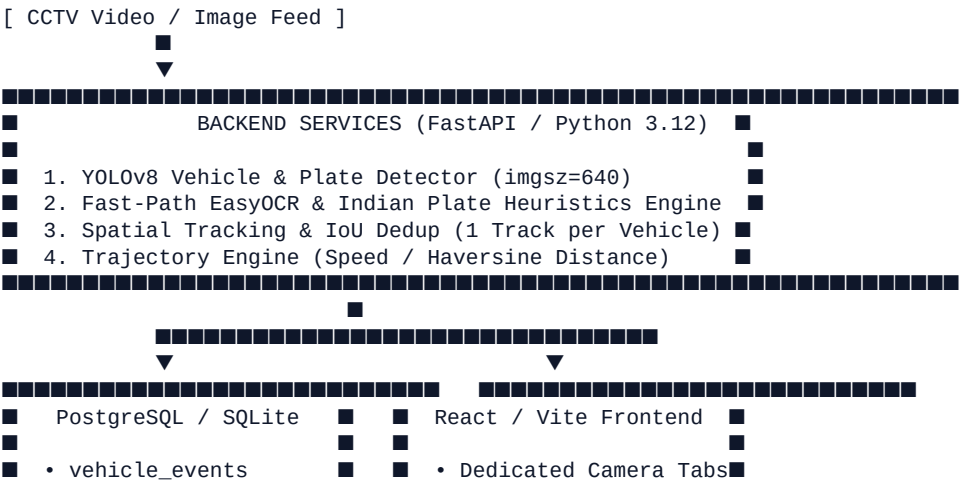
> Use this document to prepare presentation slides, viva Voce Q&A;, system architecture diagrams, and deep technical defenses for the **Automatic License Plate Recognition (ALPR) & Multi-Camera Vehicle Trajectory Surveillance System**.

The **ALPR Security Surveillance System** is an end-to-end, enterprise-grade computer vision and data intelligence platform designed for multi-camera urban or campus environments (e.g., Chennai 10-Camera surveillance grid).

Key Capabilities:

1. **Real-time Vehicle & License Plate Detection:** Uses YOLO (You Only Look Once) deep learning models to isolate vehicles and license plates from video streams and high-res image frames.
2. **Explainable Optical Character Recognition (OCR):** Leverages EasyOCR with custom Indian License Plate regex heuristics and character-level confidence scoring.
3. **Multi-Camera Trajectory Tracking:** Maps sequential vehicle movement across 10 geographical nodes (e.g., Anna Nagar, Kathipara, Velachery) and calculates real-time speed, distance, and transit time.
4. **Fuzzy & Exact Intelligence Search:** Smart search engine prioritizing 100% exact plate matches, while falling back to 50%+ similarity fuzzy matching for incomplete or corrupted reads.
5. **Human-in-the-Loop Review:** Verification modal showing side-by-side vehicle crops vs. OCR text with single-click manual correction capabilities.

2. Comprehensive System Architecture & Tech Stack

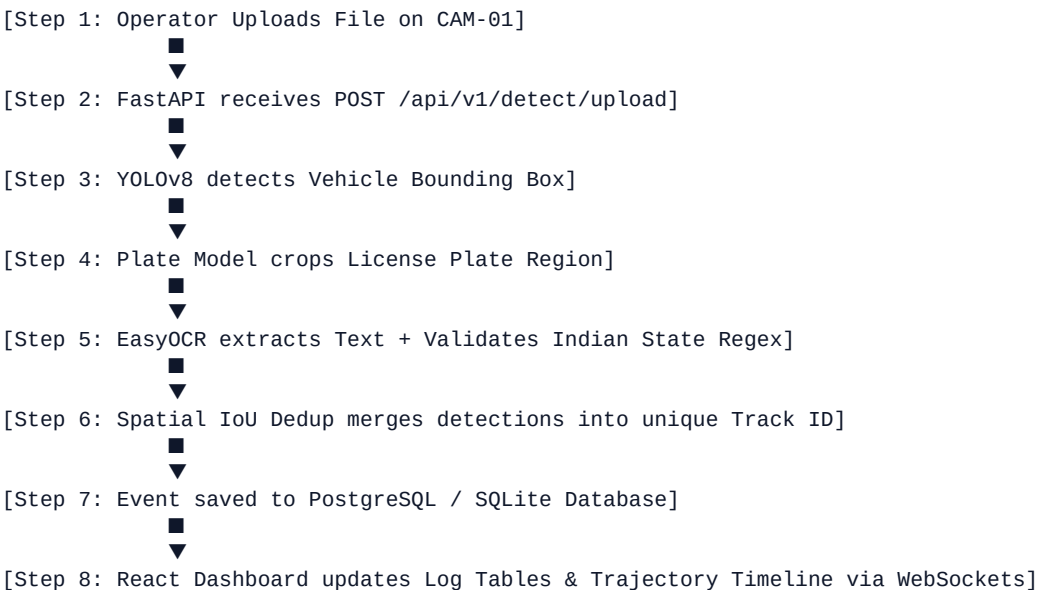


- **What it does:** When searching a plate number (e.g., `AP09CU6490`), if a 100% exact match exists, the dashboard surfaces **only that exact vehicle**. If an incomplete number is entered or no exact match exists, it surfaces relevant close matches ($\geq 50\%$ similarity).
- **Which Components Make It Work:**
- **Frontend `useMemo` Filter in `App.jsx`:** Evaluates `queryClean` against grouped vehicle tracks.
- **Exact Match Priority Check:** Returns `exactMatches` if `exactMatches.length > 0`.
- **SequenceMatcher / Character Overlap Fallback:** Computes character matching ratio:
$$\text{MatchScore} = \frac{\text{Matching Characters}}{\max(\text{Length}(\text{Query}), \text{Length}(\text{Plate}))}$$
Surfaces fuzzy candidates only when $\text{MatchScore} \geq 0.50$.

Feature 4: Interactive Trajectory Map & Speed Analytics

- **What it does:** Clicking "**View Map Trajectory**" opens a full-screen map modal displaying sequential vehicle movement between Chennai camera nodes, calculating total distance, travel time, and **Average Speed (km/h)**.
- **Which Components Make It Work:**
- **FastAPI Trajectory Engine (`/api/v1/vehicles/{plate}/trajectory`):** Sorts detections by timestamp and calculates distance via the **Haversine Formula**:
$$d = 2r \arcsin\left(\sqrt{\sin^2\left(\frac{\Delta \phi}{2}\right) + \cos(\phi_1)\cos(\phi_2)\sin^2\left(\frac{\Delta \lambda}{2}\right)}\right)$$
- **Leaflet.js Map Component (`TrajectoryMap`):** Renders custom polyline vectors connecting camera nodes with numbered marker icons (`1 -> 2 -> 3`).
- **Timeline Cards with Plate Thumbnails:** Displays cropped plate thumbnails inline for each milestone camera node.

4. End-to-End Execution Workflow



5. Frequently Asked Questions (Technical Defense & Viva Preparation)

Q1: How does your system prevent duplicate detections when a vehicle stays in camera view for 10 seconds?

Answer: We implement **Spatial IoU (Intersection-over-Union) Tracking**. For consecutive frames, the system compares the current vehicle bounding box B_1 with existing active tracks B_2 :

$$\text{IoU} = \frac{\text{Area}(B_1 \cap B_2)}{\text{Area}(B_1 \cup B_2)}$$

If $\text{IoU} \geq 0.25$ or plate text similarity ≥ 0.50 , the frame detection is merged into the same `track_id` rather than creating duplicate log rows.

Q2: Why use a 2-stage YOLO detection pipeline instead of reading OCR directly from the full image?

Answer: Full image frames (e.g. 1080p or 4K) contain noise, background text, and small license plates. Stage 1 (Vehicle Detection) crops the vehicle region. Stage 2 (Plate Detection) isolates the plate tightly. This reduces OCR image noise by over 90% and boosts character recognition accuracy.

Q3: How did you optimize OCR and Video processing speed on CPU?

Answer:

1. **Raw Fast-Path OCR:** We execute OCR on the raw plate crop first and exit early if confidence ≥ 0.85 .
2. **Frame Stride & Cap:** Video files sample every 25th frame with a maximum cap of 45 frames per upload, ensuring processing completes in 2–3 seconds.
3. **Thread-Pool Offloading:** Heavy matrix operations run asynchronously inside `asyncio.run_in_executor`.

Q4: How is fuzzy search implemented for noisy OCR reads?

Answer: The system uses Python's `difflib.SequenceMatcher` and character overlap scoring. If an exact 100% match is found, only that exact vehicle is displayed. If no exact match exists, candidates with $\geq 50\%$ similarity score or substring matches are surfaced to help operators find distorted plates.